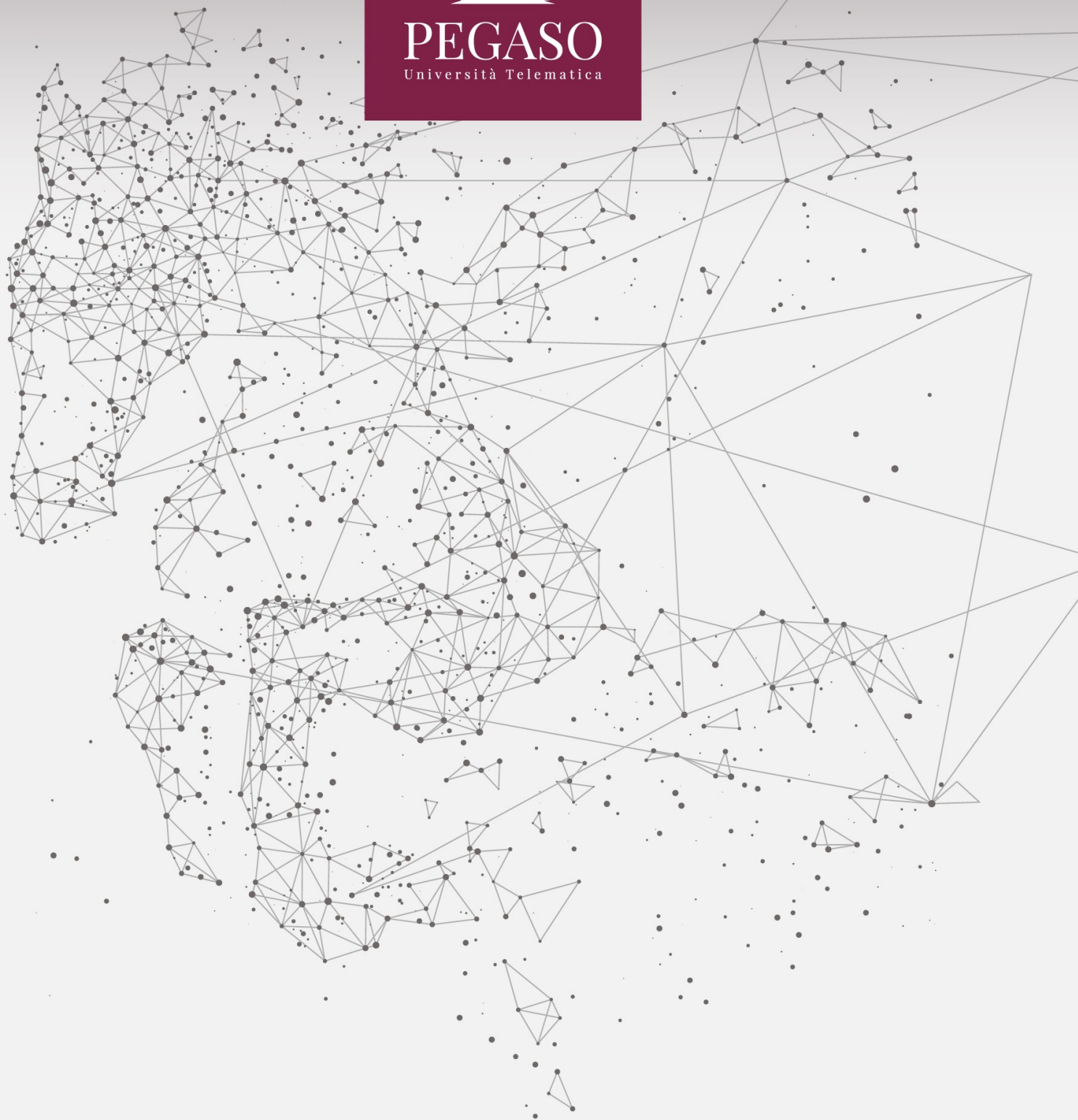




PEGASO
Università Telematica



Indice

1. ALGORITMO E RISOLUZIONE DI UN PROBLEMA	3
2. TIPOLOGIE DI FRASI DEL LINGUAGGIO.....	4
3. SEQUENZE DI ESECUZIONE	7
BIBLIOGRAFIA	10

1. Algoritmo e risoluzione di un problema

Un algoritmo è definito come una sequenza finita di passi che portano alla realizzazione di un compito, ovvero un insieme finito di istruzioni che, eseguite secondo un ordine prestabilito, permettono di giungere alla soluzione di un problema.

Nella definizione di un algoritmo, è essenziale considerare alcuni aspetti fondamentali:

1. deve essere composto da un insieme finito di passi;
2. i passi devono essere eseguiti in una sequenza precisa;
3. i dati di input devono essere elaborati per giungere alla soluzione del problema;
4. i dati di output devono essere prodotti come soluzione del problema stesso.

È importante notare che la definizione di un algoritmo è strettamente legata all'entità che dovrà interpretarlo ed eseguirlo. Infatti, gli algoritmi vengono definiti presupponendo la disponibilità di un esecutore appropriato che sarà responsabile dell'implementazione pratica delle istruzioni descritte. Quindi, sia la natura del problema che si vuole risolvere, sia le caratteristiche dell'esecutore che dovrà eseguire l'algoritmo influenzano la sua definizione.

Un programma è fondamentalmente un algoritmo scritto in un linguaggio di programmazione comprensibile all'esecutore. Il linguaggio utilizzato dev'essere tale da descrivere in modo non ambiguo tutte e sole le operazioni che l'esecutore è in grado di eseguire, fornendo così le istruzioni necessarie per formulare algoritmi interpretabili ed eseguibili.

In sostanza, si può considerare l'esecutore come il linguaggio di programmazione stesso, ignorando il suo funzionamento interno. Questo concetto consente agli sviluppatori di scrivere codice senza dover preoccuparsi dei dettagli di implementazione dell'esecutore, concentrandosi invece sulla logica e sulle procedure che definiscono il programma stesso.

La risoluzione di un problema è un processo che richiede una serie di azioni ordinate e mirate. Ecco i passi fondamentali da seguire:

1. capire preliminarmente se il problema ammette soluzioni. Questo significa raccogliere tutte le informazioni pertinenti, definire gli obiettivi da raggiungere e identificare i vincoli da tenere in considerazione;
2. nel caso ne ammetta, individuare un metodo risolutivo (algoritmo). La scelta del metodo risolutivo dipende dalla natura del problema e dalle risorse disponibili;
3. esprimere il metodo risolutivo con un linguaggio comprensibile all'esecutore a cui è rivolto.

2. Tipologie di frasi del linguaggio

In un programma, tipicamente, si possono individuare due classi fondamentali di frasi del linguaggio:

- le istruzioni, che descrivono l'esecuzione di determinate operazioni;
- le strutture di controllo, che indicano all'esecutore l'ordine in cui tali operazioni devono essere eseguite.

Le istruzioni comprendono azioni specifiche che il computer deve eseguire, come calcoli, manipolazione di dati o interazioni con l'utente. Le strutture di controllo sono utilizzate per dirigere il flusso di esecuzione del programma, determinando la sequenza in cui le istruzioni vengono eseguite. Queste includono costrutti come cicli, condizioni e salti, che permettono di creare logiche complesse e rispondere a diverse situazioni durante l'esecuzione del programma. Ad esempio, consideriamo un semplice programma che elabori un testo per garantire la correttezza delle parole utilizzate. Tale programma deve essere in grado di analizzare il testo, verificando se le parole sono scritte correttamente rispetto a un dizionario predefinito, e deve essere capace di individuare e correggere eventuali errori di sintassi.

La Fig. 1 mostra un esempio di testo dato in input a tale programma.

Esempio di testo in input

La comne osservazione dela evoluzione del mondo degli elaboratori eletronic non deve trarre in ingano. Difati a dispeto dell'evoluzione tecnologicha, l'attivit  di programmazione si propone come un'arte che ...

Figura 1 – Esempio di testo dato in input al programma per la correzione degli errori

Il programma che si vuole realizzare dovrebbe, da un lato, essere in grado di riconoscere le parole presenti nel testo che sono scritte erroneamente. Nel caso dell'esempio di input, le parole errate sono mostrate in Fig. 2.

Ricerca di eventuali errori

La **comne** osservazione **dela** evoluzione del mondo degli elaboratori **eletronic** non deve trarre in **ingano**. **Difati** a **dispeto** dell'evoluzione **tecnologic**ha, l'**attivit ** di programmazione si propone come un'arte che ...

Figura 2 – Le parole errate presenti nel testo di esempio

Una volta riconosciute le parole errate, il programma che si vuole realizzare dovrebbe modificare il testo e sostituire le parole errate con la versione corretta delle parole stesse, e produrre infine in output il testo corretto (vedi Fig. 3).

Esempio di output

La comune osservazione della evoluzione del mondo degli elaboratori elettronici non deve trarre in inganno. Difatti a dispetto dell'evoluzione tecnologica, l'attività di programmazione si propone come un'arte che ...

Figura 3 – Il testo di esempio corretto prodotto in output dal programma considerato

Un possibile soluzione per tale programma di correzione ortografica di un testo potrebbe funzionare come segue:

1. inizialmente, il programma analizza un rigo del testo;
2. **per** ogni parola del rigo **fai**:
3. **se** la parola è conosciuta
allora, controllane la corretta scrittura
altrimenti, ricerca nel vocabolario
4. **se** la parola non è scritta correttamente
allora sostituisci nel testo la parola errata con la sua correzione
5. **ripeti** le azioni da 1 a 4 per ogni rigo del testo e fino alla fine dei rigi

Le istruzioni elementari sono quelle che l'esecutore è in grado di comprendere ed eseguire direttamente senza ulteriori spiegazioni. Al contrario, le istruzioni non elementari sono istruzioni non note all'esecutore, la cui specifica deve essere resa disponibile affinché l'esecutore possa eseguire correttamente l'algoritmo.

Le istruzioni non elementari permettono di descrivere l'algoritmo con un repertorio più ricco rispetto alle sole istruzioni elementari note all'esecutore.

La specifica di un'istruzione non elementare solitamente trasforma l'istruzione stessa in un insieme di istruzioni elementari, consentendo così all'esecutore di comprendere e realizzare compiti complessi.

Le strutture di controllo di un programma includono tre tipi fondamentali:

- i costrutti di sequenza;

- i costrutti di selezione;
- i costrutti iterativi.

I costrutti di sequenza specificano che le azioni devono essere svolte una dopo l'altra, in un ordine ben definito.

I costrutti di selezione, invece, determinano che certe azioni devono essere eseguite solo se si verificano determinate condizioni, permettendo così di prendere decisioni all'interno del programma.

Infine, i costrutti iterativi definiscono azioni che devono essere ripetute un numero di volte prestabilito o fino al verificarsi di specifiche condizioni, consentendo la realizzazione di cicli che automatizzano operazioni ripetitive.

Le operazioni di un algoritmo devono possedere alcune caratteristiche fondamentali:

- la finitezza, che implica che ogni operazione deve concludersi entro un intervallo di tempo limitato dall'inizio della sua esecuzione;
- la descrivibilità, che richiede che le operazioni producano effetti chiari e osservabili, come fotografare lo stato degli oggetti coinvolti sia prima che dopo l'esecuzione;
- la riproducibilità, che assicura che le operazioni producano lo stesso effetto ogni volta che vengono eseguite nelle stesse condizioni iniziali;
- la comprensibilità, che prevede che le operazioni siano espresse in una forma che sia comprensibile per l'esecutore.

3. Sequenze di esecuzione

L'esecuzione di un algoritmo da parte di un esecutore si traduce in una successione di azioni che vengono effettuate nel tempo, evocando un processo sequenziale. Questo implica una serie di eventi che si verificano uno dopo l'altro, ciascuno con un inizio e una fine identificabili. Ogni azione dell'algoritmo segue un ordine preciso e ben definito, garantendo che il processo complessivo proceda in modo ordinato e controllato, permettendo di raggiungere il risultato desiderato passo dopo passo.

Si definisce sequenza di esecuzione, detta anche sequenza dinamica, la descrizione del processo sequenziale che elenca tutte le istruzioni eseguite, nell'ordine di esecuzione.

Si noti che un algoritmo può prescrivere più di una sequenza di esecuzione e, in alcuni casi, un numero infinito di sequenze di esecuzione. In particolare, nel caso di un processo ciclico, può accadere che il numero di sequenze sia infinito, portando a un processo che non ha mai termine. Questo accade quando le condizioni per uscire dal ciclo non vengono mai soddisfatte, causando una ripetizione indefinita delle istruzioni all'interno del ciclo.

Si chiama sequenza statica di un algoritmo la serie di istruzioni elencate nella formulazione dell'algoritmo stesso. Questa sequenza può descrivere una o più sequenze di esecuzione dinamiche differenti.

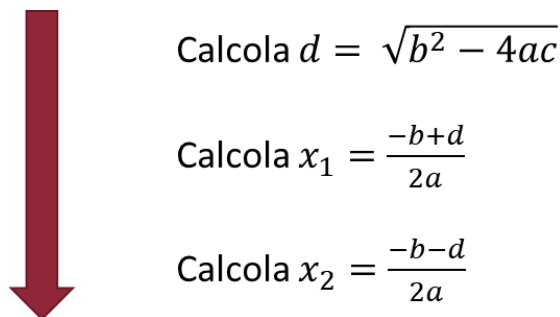


Figura 4 – Esempio di algoritmo con un'unica possibile sequenza di esecuzione

La sequenza dinamica di un algoritmo, invece, è la sequenza con cui le istruzioni sono effettivamente eseguite dall'esecutore. Il numero di sequenze dinamiche non è noto a priori e dipende dalla sequenza statica e dai dati di ingresso, poiché diverse condizioni e dati iniziali possono portare a percorsi di esecuzione differenti.

Ad esempio, nell'algoritmo riportato in Fig. 4 per il calcolo delle radici reali di un'equazione di secondo grado, esiste un'unica possibile sequenza di esecuzione.

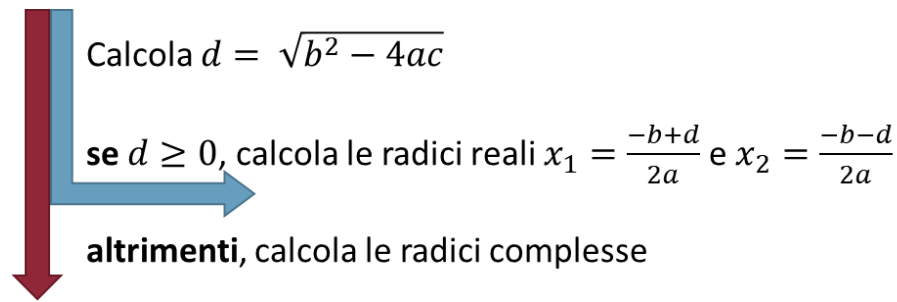


Figura 5 – Esempio di algoritmo con due possibili sequenze di esecuzione

Tuttavia, se consideriamo invece l'algoritmo riportato in Fig. 5 per il calcolo delle radici reali e complesse di un'equazione di secondo grado, osserviamo che si presentano due possibili sequenze di esecuzione:

- una per il caso in cui il discriminante sia maggiore o uguale a zero, che porta alle radici reali;
- un'altra per il caso in cui il discriminante sia minore di zero, che porta alle radici complesse coniugate.

Come ulteriore esempio, si consideri un algoritmo per lo svolgimento di un solitario di carte, come riportato in Fig. 6.

Finché ci sono carte nel mazzo, ripeti le seguenti azioni:

- mischia le carte;
- prendi quattro carte dal mazzo e disponile sul tavolo di gioco;
- **se** le quattro carte hanno tutte lo stesso valore, rimuovile dal gioco;
altrimenti, rimetti le carte nel mazzo;
- **se** il mazzo è vuoto hai vinto;
altrimenti, il gioco continua.

Figura 6 – Esempio di algoritmo per lo svolgimento di un solitario di carte

La sequenza statica di questo algoritmo descrive una moltitudine di sequenze dinamiche diverse che possono verificarsi a seconda dei risultati prodotti dalle azioni dell'algoritmo e, in altre parole, di come procede lo svolgimento del solitario.

Sequenza di esecuzione 1 (si eliminano carte al primo tentativo):

- mischiate le carte;
- pescate quattro carte dal mazzo;
- le quattro carte hanno tutte lo stesso valore, si eliminano dal gioco;
- il mazzo contiene ancora carte, e il gioco continua.

Figura 7 – Sequenza di esecuzione che si verifica nel caso si riesca a scartare carte già al primo tentativo

Ad esempio, se analizziamo il caso in cui si riesca effettivamente a scartare delle carte, ciò può accadere già al primo tentativo come riportato in Fig. 7.

Sequenza di esecuzione 2 (si eliminano carte al secondo tentativo):

- mischiate le carte;
- pescate quattro carte dal mazzo;
- le quattro carte non hanno lo stesso valore, si rimettono nel mazzo;
- il mazzo contiene ancora carte, e il gioco continua;
- mischiate le carte;
- pescate quattro carte dal mazzo;
- le quattro carte hanno tutte lo stesso valore, si eliminano dal gioco;
- il mazzo contiene ancora carte, e il gioco continua.

Figura 8 – Sequenza di esecuzione del caso in cui si riesca a scartare carte soltanto al secondo tentativo

Se invece si riesca a scartare le carte soltanto al secondo tentativo, la sequenza di esecuzione cambia e diventa quella riportata in Fig. 8.

Sequenza di esecuzione 4 (non si eliminano mai carte):

- si è ripetuto infinite volte senza aver eliminato carte
 - mischiate le carte;
 - pescate quattro carte dal mazzo;
 - le quattro carte non hanno lo stesso valore, si rimettono nel mazzo;
 - il mazzo contiene ancora carte, e il gioco continua;

Figura 9 – Sequenza di esecuzione del caso in cui non si riesca mai a scartare carte

Altra sequenza di esecuzione è quella che descrive il caso in cui non si riesce mai a scartare carte, e l'algoritmo procede all'infinito come riportato in Fig. 9.

Bibliografia

- Le radici dell'informatica. Dal bit alla programmazione strutturata. Angelo Chianese, Vincenzo Moscato, Antonio Picariello, Carlo Sansone. Maggioli Editore.